LAPORAN PRAKTIKUM

STRUKTUR DATA

IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)* JAM

DALAM PEMROGRAMAN *JAVA*

Disusun Oleh:

Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Wahyudi. Dr. S.T.M.T

Asisten Praktikum:
Muhammad Galid

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan Syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas Rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 1 dengan judul "Implementasi *Abstract Data Type (ADT)* Jam dalam Pemrograman Java" tepat pada waktunya.

Penyusunan laporan ini bertujuan untuk memenuhi salah satu tugas dalam mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran bagi penulis dalam memahami konsep *Abstract Data Type (ADT)* serta penerapannya dalam pemrograman Java. Melalui praktikum ini, penulis dapat mempelajari bagaimana merepresentasikan data waktu (jam) ke dalam sebuah objek serta mengimplementasikan berbagai operasi seperti validasi waktu, konversi ke detik, operasi aritmatika (penjumlahan dan pengurangan), serta operasi relasional (perbandingan antar waktu).

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat terbuka terhadap kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Ucapan terima kasih yang sebesar-besarnya penulis sampaikan kepada dosen pengampu, asisten praktikum, serta semua pihak yang terlibat dalam pelaksanaan praktikum dan penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi penulis maupun pembaca.

Padang, 31 Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 5
BAB II PEMBAHASAN 6
2.1 Dasar Teori 6
2.1.1 Abstract Data Type (ADT). 6
2.1.2 Konsep Kerja Abstract Data Type (ADT) 6
2.1.3* Komponen dalam *Abstract Data Type (ADT) 7
2.1.4 Primitif Type* dalam *Abstract Data Type (ADT) 8
2.2 Langkah Kerja 10
2.2.1 Pembuatan *Package* dan *Class* 10
2.2.2 Membuat ADT Jam_2511532002 14
2.2.3 Membuat Program JamDriver 16
2.2.4 Membuat Program JamDriver2 17
2.2.5 *Upload* ke GitHub 19
2.3 Hasil 21
2.3.1 Hasil ADT Jam_2511532002 21
2.3.2 Hasil JamDriver 22
2.3.3 Hasil JamDriver2 22
BAB III KESIMPULAN 24
3.1 Kesimpulan 24
3.2 Saran 24
DAFTAR PUSTAKA 25

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan pemrograman, pengolahan data yang terstruktur sangat penting untuk menghasilkan program yang efisien, terorganisir, dan mudah dikembangkan. Salah satu konsep yang digunakan dalam struktur data adalah *Abstract Data Type (ADT*), yaitu suatu tipe data abstrak yang mendefinisikan data beserta operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan implementasi internalnya.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami cara merancang dan mengimplementasikan *Abstract Data Type (ADT)* menggunakan bahasa pemrograman Java, mulai dari pembuatan tipe data objek (*class*), penentuan atribut seperti jam, menit, dan detik, hingga implementasi berbagai operasi seperti validasi, konversi ke detik, operasi aritmatika, serta operasi relasional untuk mengatur interaksi dan pengolahan data waktu dalam program.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1.2.1 Memahami konsep dasar *Abstract Data Type (ADT)* dalam struktur data.

1.2.2 Mampu mengimplementasikan *Abstract Data Type (ADT)* dalam bahasa pemrograman *Java.*

1.2.3 Mampu membuat *ADT* Jam beserta operasi-operasinya seperti validasi, konversi, aritmatika, dan relasional

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum ini antara lain:

1.3.1 Menambah pemahaman mengenai konsep *ADT* dalam pemrograman.

1.3.2 Meningkatkan kemampuan dalam mengimplementasikan struktur data secara terstruktur.

1.3.3 Membantu dalam memahami pengolahan data waktu menggunakan pendekatan pemrograman.

1.3.4 Menjadi dasar dalam pengembangan program yang lebih kompleks di masa yang akan datang.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan menyimpan data agar dapat diakses dan diolah secara efisien. Dalam pengembangan program, diperlukan suatu cara untuk merepresentasikan data beserta operasi-operasi yang dapat dilakukan terhadapnya secara terstruktur. Salah satu pendekatan yang digunakan adalah *Abstract Data Type (ADT).*

ADT memungkinkan programmer untuk fokus pada fungsi dan perilaku suatu data tanpa harus langsung memikirkan bagaimana data tersebut diimplementasikan di dalam memori. Dengan demikian, penggunaan *ADT* dapat meningkatkan modularitas, keterbacaan, serta kemudahan dalam pengembangan dan pemeliharaan program nantinya.

2.1.1 Abstract Data Type (ADT).

Abstract Data Type (ADT) merupakan suatu konsep yang mendefinisikan sebuah tipe data beserta sekumpulan operasi (primitif) yang dapat dilakukan terhadap tipe data tersebut tanpa memperhatikan bagaimana implementasi internalnya . Dengan kata lain, *ADT* lebih menekankan pada apa yang dapat dilakukan oleh suatu data, bukan bagaimana data tersebut direpresentasikan secara fisik di dalam memori.

2.1.2 Konsep Kerja Abstract Data Type (ADT)

Konsep kerja ADT bersifat modular, di mana ADT berperan sebagai *supplier* (penyedia) yang menyediakan tipe data dan operasi-operasinya, sedangkan program lain bertindak sebagai *client* (pengguna) yang memanfaatkan ADT tersebut.

a) *Supplier* (penyedia)
   Menyediakan tipe data dan operasi-operasinya
b) *Client* (pengguna)
   Menggunakan *ADT* tanpa perlu mengetahui detail implementasinya

Hal ini memungkinkan pemisahan yang jelas antara spesifikasi dan implementasi, sehingga program menjadi lebih mudah dikembangkan, diuji, dan dipelihara.

Dalam implementasinya, ADT biasanya dibagi menjadi:

1) Spesifikasi
   Berisi definisi tipe data dan operasi
2) *Body*/Implementasi
   Berisi kode program dari operasi
3) Driver Program
   Digunakan untuk menguji *ADT*

2.1.3 Komponen dalam *Abstract Data Type (ADT)*

Sebuah *ADT* umumnya terdiri dari beberapa komponen utama, yaitu:

a) Tipe Data (*Type*)
   Merupakan definisi struktur data yang digunakan, misalnya record atau class yang berisi atribut-atribut tertentu.
b) *Primitif* (Operasi Dasar)Primitif adalah operasi-operasi yang dapat dilakukan terhadap ADT, meliputi:
   1) Konstruktor/Kreator
      Untuk membuat objek.
   2) Selektor (*Getter*)
      Untuk mengambil nilai atribut
   3) Mutator (*Setter*)
      Untuk mengubah nilai atribut

4) Validator

Untuk memastikan data valid

5) Operator Relasional

Untuk membandingkan dua objek

6) Operator Aritmatika

Untuk melakukan perhitungan

7) Konversi

Untuk mengubah tipe data ke bentuk lain

c) *Invariant*

Aturan atau batasan yang harus selalu dipenuhi oleh suatu objek agar tetap valid.

d) Aksioma

Aksioma merupakan aturan atau definisi formal yang menjelaskan hubungan logis antar operasi dalam *ADT*. Aksioma digunakan untuk memastikan bahwa setiap operasi yang dilakukan memberikan hasil yang konsisten dan sesuai dengan spesifikasi.

2.1.4 Primitif Type dalam *Abstract Data Type (ADT)*

Primitif type dalam *ADT* adalah sekumpulan operasi dasar yang didefinisikan untuk memanipulasi tipe data tersebut. Primitif ini biasanya diimplementasikan dalam bentuk fungsi atau prosedur.

Beberapa jenis primitif yang umum digunakan antara lain:

1) *IsValid*

Untuk mengecek validitas data.

2) *Make/Create*

Untuk membentuk objek baru.

3) *Get*

Untuk mengambil nilai komponen.

4) *Set*

Untuk mengubah nilai komponen.

5) *Compare*

   Untuk membandingkan dua objek.

6) Operasi tambahan

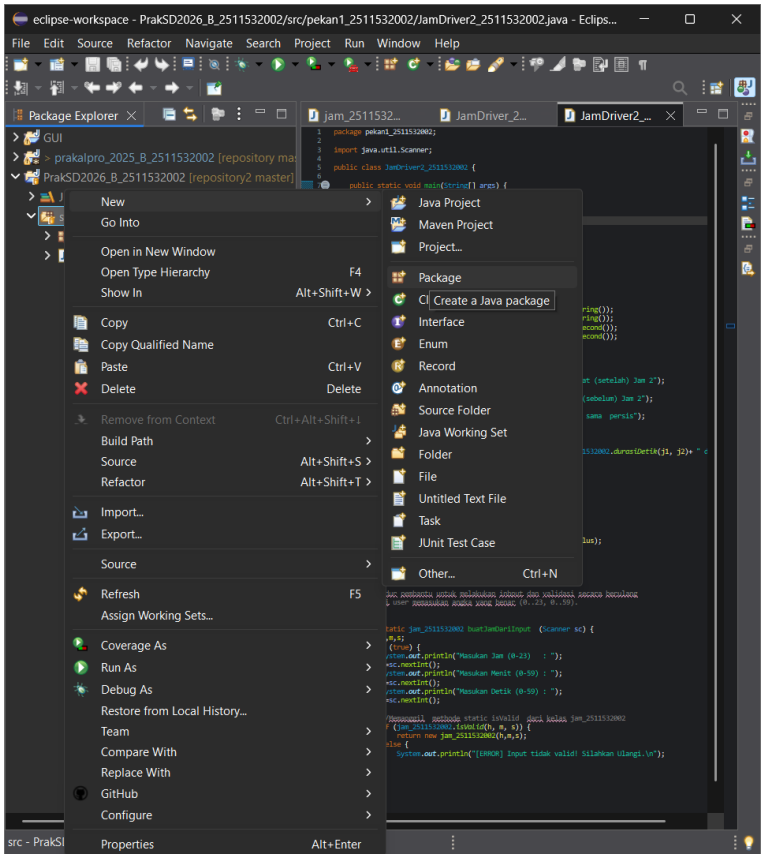
   Seperti penjumlahan, pengurangan, atau manipulasi lainnya sesuai kebutuhan.

Primitif-primitif ini menjadi inti dari sebuah *ADT* karena menentukan bagaimana data dapat digunakan dan dimanipulasi dalam program.

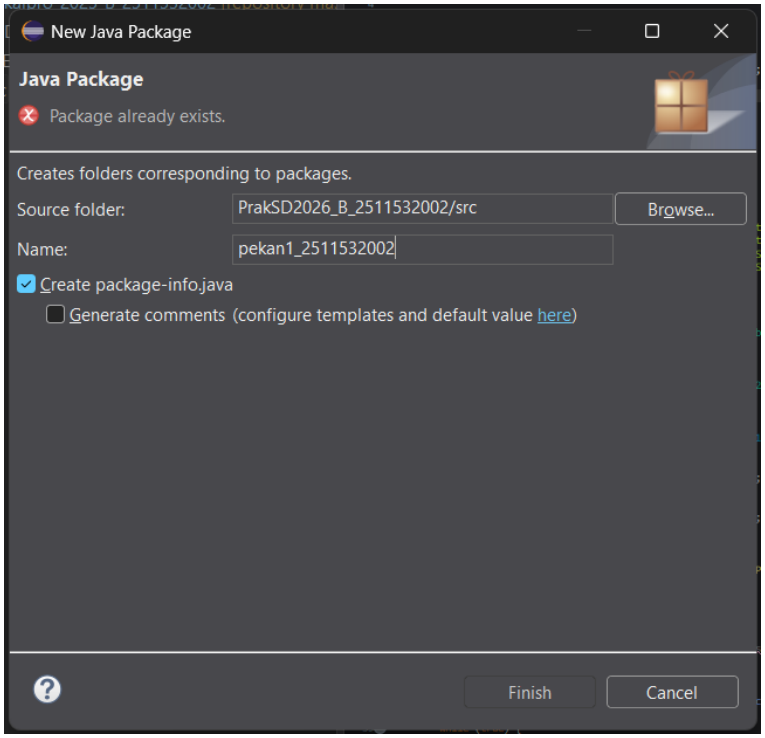
2.2 Langkah Kerja

2.2.1 Pembuatan *Package* dan *Class*

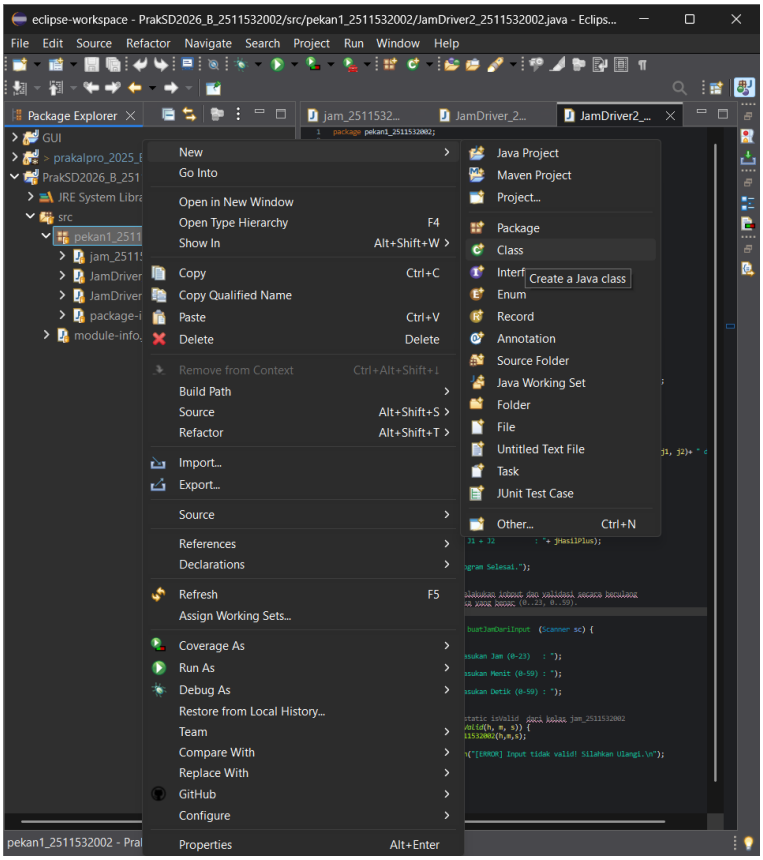
1) Sebelum membuat suatu program *user* perlu untuk membuat *package* terlebih dahulu. Klik kanan pada *src* pada *repository* *"PrakSD2026_B_2511532002"* lalu pilih "*New*" dan klik "*Package*".



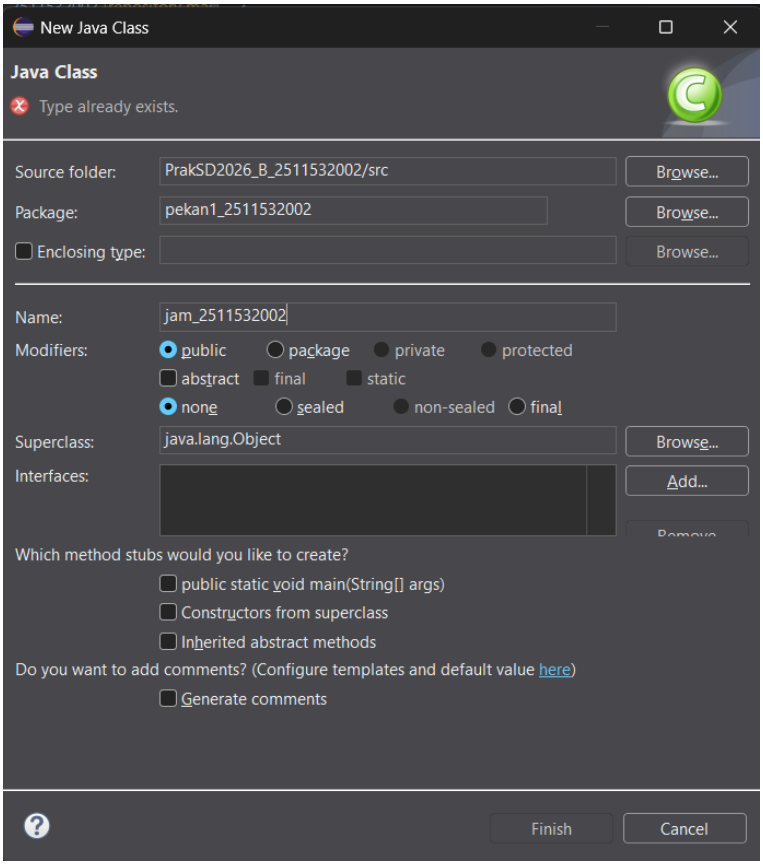
2) Setelah itu akan muncul *pop up* "*Java Package*", buat nama *package* dengan ketentuan tanpa *capslock, space*, dan karakter khusus lainnya. Lalu klik "*finnish*".



3) Selajutnya adalah membuat membuat *class*. pada package yang telah dibuat tapi klik kanan lalu pilih "*New*" dan klik "*Class*".



4) Setelah itu pada menu "*Java Class*" buat nama *class*, dengan ketentuan nama harus capslock diawal kalimat dan tanpa spasi. Lalu pilih "Public static void main (*String[] grs*)" agar tidak perlu membuat methode *main()* secara manual. Selanjutnya "*finish*".



2.2.2 Membuat ADT Jam_2511532002

1. Setelah membuat *class jam_2511532002,* maka mahasiswa dapat menulis *syntax* nya.
2. Mendefinisikan atribut (*Type)* dan fungsi validator nya yaitu "*isValid"* untuk memastikan nilai jam, menit, dan detik berada dalam rentang yang valid.

```java
package pekan1_2511532002;

public final class jam_2511532002 {
    private int hh; //o.23
    private int mm; //0.59
    private int ss; //0.59

    public static boolean isValid (int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }
    // --- Konstruktor/ Kreator ---
    public jam_2511532002 (int h, int m, int s) {
        this.hh = h; this.mm =m; this.ss =s;
    }

```

Keterangan:

- Atribut (*Type*)
  Menentukan atribut *hh, mm,* dan *ss* sebagai komponen utama yang merepresentasikan waktu dalam format jam, menit, dan detik.

- *Validator*
  Mengimplementasikan *method isValid()* untuk memastikan nilai jam, menit, dan detik berada dalam rentang yang valid.

3. Membuat konstruktor
   Menyediakan konstruktor untuk membentuk objek jam berdasarkan nilai jam, menit, dan detik yang diberikan.

```java
// --- Konstruktor/ Kreator ---
public jam_2511532002 (int h, int m, int s) {
    this.hh = h; this.mm =m; this.ss =s;
}
```

4. Membuat Selektor (*Getter*)
   Mengimplementasikan method seperti *getHour(), getMinute(),* dan *getSecond()* untuk mengambil nilai dari masing-masing atribut.

```
// --- Selektor ----
public int getHour() {return hh; }
public int getMinute() {return mm; }
public int getSecond() {return ss; }

// --- Mutator ---
public void setHour(int h) {this.hh=h;}
public void setMinute(int m) {this.mm=m;}
public void setSecond(int s) {this.ss=s;}
```

5. Membuat Mutator (*Setter)*
   Mengimplementasikan method *setHour(), setMinute(),* dan *setSecond()* untuk mengubah nilai atribut jam.

```
// --- Mutator ---
public void setHour(int h) {this.hh=h;}
public void setMinute(int m) {this.mm=m;}
public void setSecond(int s) {this.ss=s;}
```

6. Mengkonversi dengan membuat *methode toSecond()* dan *fromSeconds()* untuk mengubah representasi waktu ke dalam satuan detik dan sebaliknya.

```
// --- Konversi ---
public int toSecond () {return hh*3600 + mm*60 + ss;}
public static jam_2511532002 fromSeconds (int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %= 24*3600; //wrap 24 jam
    int h = total/3600; total %= 3600;
    int m = total/60; int s=total&60;
    return new jam_2511532002(h,m,s);
}
```

7. Mengimplementasikan operasi relasional. Membuat *method* seperti *compareTo()* dan *equals()* untuk membandingkan dua objek jam.

```
// --- Relational ---
public int compareTo (jam_2511532002 other) {return Integer.compare(this.toSecond(), other.toSecond()); }
public boolean equals (Object o) {
    if (!(o instanceof jam_2511532002 j)) return false;
    return hh == j.hh & mm == j.mm & ss == j.ss;
}
public int hashCode () { return java.util.Objects.hash(hh, mm, ss); }
```

8. Mengimplementasikan operasi aritmatika. Menambahkan *method* seperti *plus*(), *minus*(), *nextSecond*(), dan *prevSecond*() untuk melakukan operasi perhitungan waktu.

```
// --- Aritmatika ---
public jam_2511532002 plus(jam_2511532002 other) {return fromSeconds (this.toSecond() + other.toSecond());}
public jam_2511532002 minus(jam_2511532002 other) {return fromSeconds(Math.floorMod(this.toSecond() - other.toSecond(), 24*(3600))); }
public jam_2511532002 nextSecond() {return fromSeconds (this.toSecond() + 1); }
public jam_2511532002 nextNSeconds (int n) {return  fromSeconds(this.toSecond() + Math.max(0, n));}
public jam_2511532002 prevSecond() {return  fromSeconds(Math.floorMod(this.toSecond() -1, 24*3600)); }
public jam_2511532002 prevNSeconds(int n) {return fromSeconds (Math.floorMod (this.toSecond()-Math.max(0,n), 24*3600)); }
```

9. Membuat operasi tambahan untuk duasi atau *method durasiDetik()* dan *methode toString()* untuk mengubah objek jam menjadi bentuk *string* dengan format *hh:mm:ss*.

```
// --- Durasi (detik) ---  bisa negatif jika this > other?  sesuai spesifikasi  durasi (Jaw, Jakh)
public static int durasiDetik(jam_2511532002 jaw, jam_2511532002 jakh) {return  jakh.toSecond() - jaw.toSecond(); }
//overriding ke String
public String toString( ) {return String.format("%02d: %02d:%02d", hh,mm,ss); }
}
```

2.2.3 Membuat Program JamDriver

Setelah membuat program *ADT* nya yaitu *"jam_2511532002"* sebagai *supplier* nya sekarang kita membuat program *client* untuk menguji atau menggunakan *ADT* yang dibuat.

Untuk menguji, tulis *syntax* berikut.

```
*jam_2511532002.java   JamDriver_2511532002.java   JamDriver2_2511532002.java
1  package pekan1_2511532002;
2
3  public class JamDriver_2511532002 {
4
5      public static void main(String[] args) {
6          jam_2511532002 a = new jam_2511532002 (23, 59, 50);
7          jam_2511532002 b = new jam_2511532002 (0, 0, 15);
8          System.out.println("a                 = "+a);
9          System.out.println("b                 = "+b);
10         System.out.println("a + b             = "+a.plus(b));
11         System.out.println("next 20s          = " + a.nextNSeconds (20));
12         System.out.println("durasi (a, b)     = " + jam_2511532002.durasiDetik(a, b));
13         System.out.println("a .compareTo (b) = "+ a.compareTo (b));
14     }
15
16 }
```

Keterangan:

1. *jam_2511532002 x = new jam_2511532002 (xx, xx, xx);*
   Syntax ini digunakan untuk membuat objek jam.

2. *System.out.println("x = "+x);*
   Digunakan untuk menampilkan nilai objek x ke layar menggunakan *method toString()*.

3. *System.out.println(x + y = "+x.plus(y));*
   Digunakan untuk melakukan operasi penjumlahan antara dua objek jam.

4. *x.nextNSeconds(20)*
   Digunakan untuk menambahkan 20 detik pada objek jam x.

5. *jam_2511532002.durasiDetik(x, y)*
   Digunakan untuk menghitung selisih waktu antara jam x dan y dalam satuan detik.

6. *a.compareTo(b)*
   Digunakan untuk membandingkan dua objek jam.

2.2.4 Membuat Program JamDriver2

Setelah membuat program ADT nya yaitu "*jam_2511532002*" sebagai *supplier* nya sekarang kita membuat program *client* untuk menguji atau menggunakan *ADT* yang dibuat.

Untuk menguji, tulis *syntax* berikut.

*jam_2511532002.java | JamDriver_2511532002.java | JamDriver2_2511532002.java

```java
package pekan1_2511532002;

import java.util.Scanner;

public class JamDriver2_2511532002 {

    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Program Driver Objek Jam ===");

        //1. Input Jam Pertama
        System.out.println("\n --- Input Jam 1 ---");
        jam_2511532002 j1 = buatJamDariInput (input);

        //2. Input Jam Kedua
        System.out.println("\n --- Input Jam 2 ---");
        jam_2511532002 j2 = buatJamDariInput (input);

        // 3. Menampilkan Data
        System.out.println("\n ---  Hasil Operasi ");
        System.out.println(" Jam 1 (String)        : " +j1.toString());
        System.out.println(" Jam 2 (String)        : " +j2.toString());
        System.out.println(" Jam 1 dalam detik     : " + j1.toSecond());
        System.out.println(" Jam 2 dalam detik     : " + j2.toSecond());

        //4.  Operasi Relational  (Perbandingan)
        int perbandingan = j1.compareTo (j2);
        if (perbandingan > 0) {
            System.out.println("Status       : Jam 1 lebih lambat (setelah) Jam 2");
        }else if (perbandingan < 0) {
            System.out.println("Status       : Jam 1 ebih awal (sebelum) Jam 2");
        }else {
            System.out.println("Status       : Jam 1  dan Jam 2 sama  persis");
        }

        //5.  Operasi Aritmatika
        System.out.println(" Durasi  (J1 ke J2)    : " +jam_2511532002.durasiDetik(j1, j2)+ " detik");

        jam_2511532002 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya : "+ jNext);

        jam_2511532002 jPrev= j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya : "+ jPrev);

        //6. Operasi  Penjumlahan Jam
        jam_2511532002 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2          : "+ jHasilPlus);

        input.close();
        System.out.println("\n Program Selesai.");
    }
    /**
     *  Prosedur pembantu untuk melakukan input dan validasi secara berulang
     *  sampai user memasukan angka yang benar (0..23, 0..59).
     */

    private static jam_2511532002 buatJamDariInput  (Scanner sc) {
        int h,m,s;
        while (true) {
            System.out.println("Masukan Jam (0-23)   : ");
            h=sc.nextInt();
            System.out.println("Masukan Menit (0-59) : ");
            m=sc.nextInt();
            System.out.println("Masukan Detik (0-59) : ");
            s=sc.nextInt();

            //Memanggil  metode static isValid  dari kelas jam_2511532002
            if (jam_2511532002.isValid(h, m, s)) {
                return new jam_2511532002(h,m,s);
            }else {
                System.out.println("[ERROR] Input tidak valid! Silahkan Ulangi.\n");
            }
        }


    }
```

Keterangan:

1. *Scanner input = new Scanner(System.in);*
   Digunakan untuk menerima *input* dari pengguna melalui keyboard.

2. *jam_2511532002 x = buatJamDariInput(input);*
   Digunakan untuk membuat objek jam berdasarkan *input* pengguna.

3. *while (true) { ... }*
   Digunakan untuk melakukan perulangan hingga pengguna memasukkan data yang valid.

4. *if (jam_2511532002.isValid(h, m, s))*
5. Digunakan untuk memvalidasi apakah input jam, menit, dan detik sesuai dengan batas yang ditentukan.

6. *return new jam_2511532002(h, m, s);*
   Digunakan untuk mengembalikan objek jam yang telah valid.

7. *x.toString()*
   Digunakan untuk mengubah objek jam menjadi format *string (hh:mm:ss*).

8. *x.toSecond()*
   Digunakan untuk mengkonversi waktu ke dalam satuan detik.

9. *int hasil = x1.compareTo(x2);*
   Digunakan untuk menyimpan hasil perbandingan dua objek jam.

10. *if (hasil > 0) ... else if (hasil < 0) ... else ...*
    Digunakan untuk menentukan kondisi apakah waktu lebih besar, lebih kecil, atau sama.

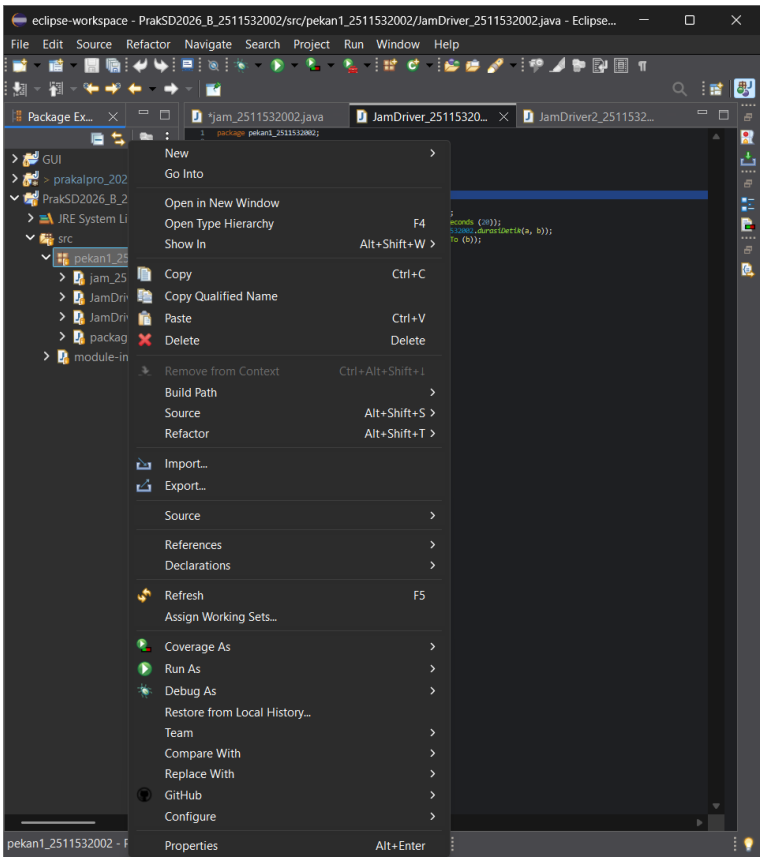
11. *x.nextSecond() dan x.prevSecond()*
    Digunakan untuk mendapatkan waktu berikutnya dan sebelumnya (±1 detik).

12. *x1.plus(x2)*
    Digunakan untuk menjumlahkan dua objek jam.

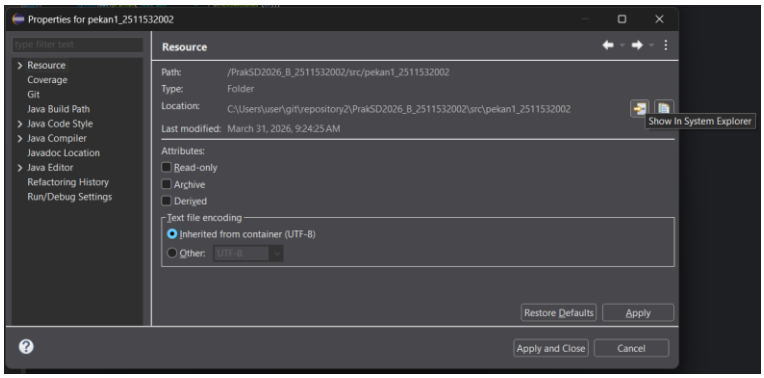
13. *input.close();*
    Digunakan untuk menutup objek *input* setelah selesai digunakan.

2.2.5 *Upload* ke GitHub

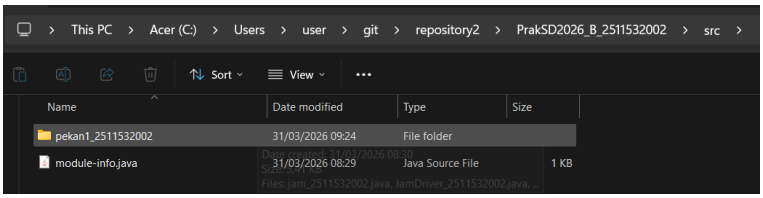
1. Setelah semua program selesai dibuat, selanjutnya adalah *user* perlu untuk memasukan program yang dibuat di Eclips ke *repository* GitHub. Selain dengan cara *"Team"* atau *"Commit and Push"*. Bisa juga dengan cara *upload* manual. Caranya klik kanan pada *package,* lalu pilih *properties.*



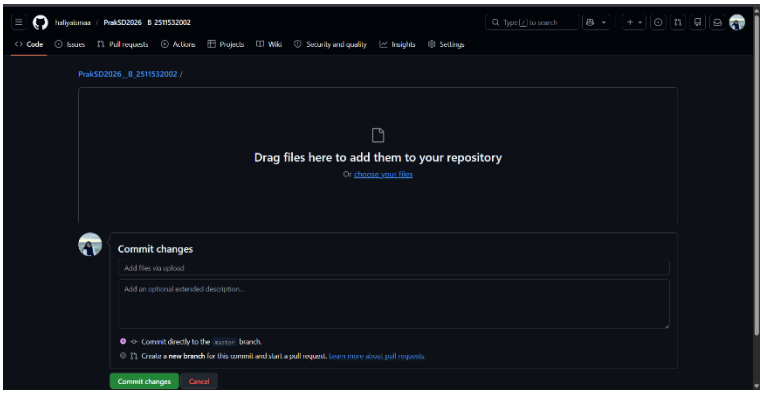
2. Maka akan muncul *pop up menu "Properteis for pekan1_2511532002"*. Lalu klik *icon "Show in System Folder"*.



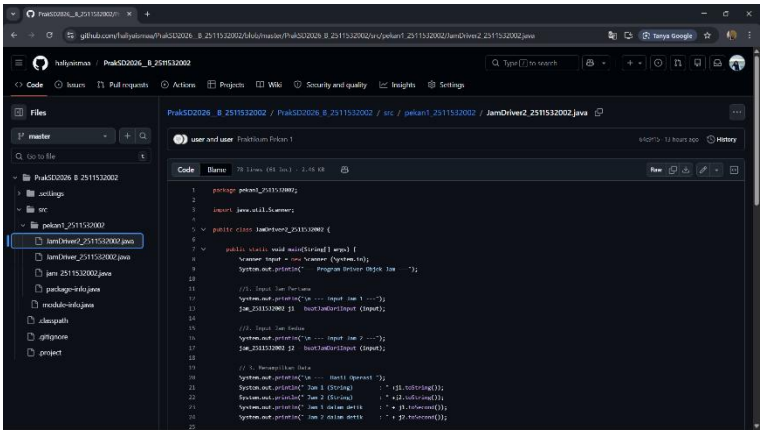
3. Maka akan menunjukan lokasi *file "pekan1_2511532002"* pada *File Explorer computer.*



4. Setelah itu *drag and drop file "pekan1_2511532002"* ke GitHub. Lalu tambahkan keterangan jika perlu. Setelah itu klik *upload.*



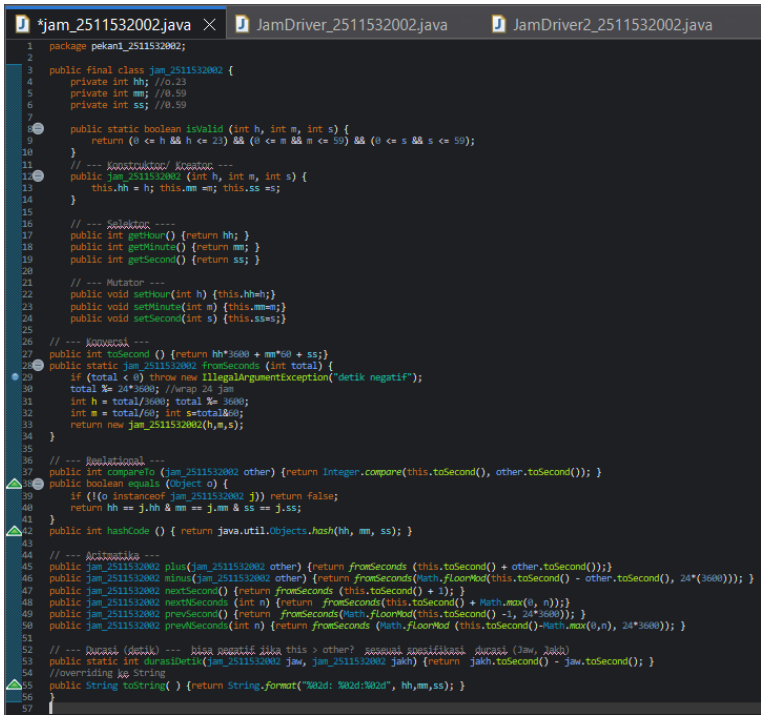
5. Maka dapat dilihat pada *repository GitHub* bahwa file "*peka8_2511532002*" telah terunggah via *Upload.*
   Dapat kita lihat disini jika kita buka, maka akan muncul *file package* dan *class* yang telah di buat di Eclipse.



2.3 Hasil

2.3.1 Hasil ADT Jam_2511532002

Berikut merupakan hasil akhir dari *ADT jam_2511532002.*



```java
*jam_2511532002.java   JamDriver_2511532002.java   JamDriver2_2511532002.java
package pekan1_2511532002;

public final class jam_2511532002 {
    private int hh; //0.23
    private int mm; //0.59
    private int ss; //0.59

    public static boolean isValid (int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }
    // --- Konstruktor/ Kreator ---
    public jam_2511532002 (int h, int m, int s) {
        this.hh = h; this.mm =m; this.ss =s;
    }

    // --- Selektor ---
    public int getHour() {return hh; }
    public int getMinute() {return mm; }
    public int getSecond() {return ss; }

    // --- Mutator ---
    public void setHour(int h) {this.hh=h;}
    public void setMinute(int m) {this.mm=m;}
    public void setSecond(int s) {this.ss=s;}

// --- Konversi ---
public int toSecond () {return hh*3600 + mm*60 + ss;}
public static jam_2511532002 fromSeconds (int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %= 24*3600; //wrap 24 jam
    int h = total/3600; total %= 3600;
    int m = total/60; int s=total&60;
    return new jam_2511532002(h,m,s);
}

// --- Relational ---
public int compareTo (jam_2511532002 other) {return Integer.compare(this.toSecond(), other.toSecond()); }
public boolean equals (Object o) {
    if (!(o instanceof jam_2511532002 j)) return false;
    return hh == j.hh & mm == j.mm & ss == j.ss;
}
public int hashCode () { return java.util.Objects.hash(hh, mm, ss); }

// --- Aritmatika ---
public jam_2511532002 plus(jam_2511532002 other) {return fromSeconds (this.toSecond() + other.toSecond());}
public jam_2511532002 minus(jam_2511532002 other) {return fromSeconds(Math.floorMod(this.toSecond() - other.toSecond(), 24*(3600))); }
public jam_2511532002 nextSecond() {return fromSeconds (this.toSecond() + 1); }
public jam_2511532002 nextNSeconds (int n) {return  fromSeconds(this.toSecond() + Math.max(0, n));}
public jam_2511532002 prevSecond() {return  fromSeconds(Math.floorMod(this.toSecond() -1, 24*3600)); }
public jam_2511532002 prevNSeconds(int n) {return fromSeconds (Math.floorMod (this.toSecond()-Math.max(0,n), 24*3600)); }

// --- durasi (Jakik) ---  hitung selisih jika this > other?  selisih positif  durasi (Jaw, Jakh)
public static int durasiDetik(jam_2511532002 jaw, jam_2511532002 jakh) {return  jakh.toSecond() - jaw.toSecond(); }
//overriding ke String
public String toString( ) {return String.format("%02d: %02d:%02d", hh,mm,ss); }
}
```

2.3.2 Hasil JamDriver

Berikut merupakan hasil akhir dari program *JamDriver.*

```java
package pekan1_2511532002;

public class JamDriver_2511532002 {

    public static void main(String[] args) {
        jam_2511532002 a = new jam_2511532002 (23, 59, 50);
        jam_2511532002 b = new jam_2511532002 (0, 0, 15);
        System.out.println("a                 = "+a);
        System.out.println("b                 = "+b);
        System.out.println("a + b             = "+a.plus(b));
        System.out.println("next 20s          = " + a.nextNSeconds (20));
        System.out.println("durasi (a, b)     = " + jam_2511532002.durasiDetik(a, b));
        System.out.println("a .compareTo (b) = "+ a.compareTo (b));
    }

}
```

```
Console
<terminated> JamDriver_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.f
a                 = 23: 59:50
b                 = 00: 00:15
a + b             = 00: 00:04
next 20s          = 00: 00:08
durasi (a, b)     = -86375
a .compareTo (b) = 1
```

2.3.3 Hasil JamDriver2

Berikut merupakan hasil akhir dari program *JamDriver2.*

```java
package pekan1_2511532002;

import java.util.Scanner;

public class JamDriver2_2511532002 {

    public static void main(String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Program Driver Objek Jam ===");

        //1. Input Jam Pertama
        System.out.println("\n --- Input Jam 1 ---");
        jam_2511532002 j1 = buatJamDariInput (input);

        //2. Input Jam Kedua
        System.out.println("\n --- Input Jam 2 ---");
        jam_2511532002 j2 = buatJamDariInput (input);

        // 3. Menampilkan Data
        System.out.println("\n ---  Hasil Operasi ");
        System.out.println(" Jam 1 (String)        : " +j1.toString());
        System.out.println(" Jam 2 (String)        : " +j2.toString());
        System.out.println(" Jam 1 dalam detik     : " + j1.toSecond());
        System.out.println(" Jam 2 dalam detik     : " + j2.toSecond());

        //4.  Operasi Relational  (Perbandingan)
        int perbandingan = j1.compareTo (j2);
        if (perbandingan > 0) {
            System.out.println("Status       : Jam 1 lebih lambat (setelah) Jam 2");
        }else if (perbandingan < 0) {
            System.out.println("Status       : Jam 1 ebih awal (sebelum) Jam 2");
        }else {
            System.out.println("Status       : Jam 1  dan Jam 2 sama  persis");
        }

        //5.  Operasi Aritmatika
        System.out.println(" Durasi  (J1 ke J2)    : " +jam_2511532002.durasiDetik(j1, j2)+ " detik");

        jam_2511532002 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya : "+ jNext);

        jam_2511532002 jPrev= j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya : "+ jPrev);

        //6. Operasi  Penjumlahan Jam
        jam_2511532002 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2          : "+ jHasilPlus);

        input.close();
        System.out.println("\n Program Selesai.");
    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * oleh user menggunakan nilai jam (0..23, 0..59).
     */

    private static jam_2511532002 buatJamDariInput  (Scanner sc) {
        int h,m,s;
        while (true) {
            System.out.println("Masukan Jam (0-23)   : ");
            h=sc.nextInt();
            System.out.println("Masukan Menit (0-59) : ");
            m=sc.nextInt();
            System.out.println("Masukan Detik (0-59) : ");
            s=sc.nextInt();

            //Memanggil  metode static isValid  dari kelas jam_2511532002
            if (jam_2511532002.isValid(h, m, s)) {
                return new jam_2511532002(h,m,s);
            }else {
                System.out.println("[ERROR] Input tidak valid! Silahkan Ulangi.\n");
            }
        }


    }

}
```

```
Console
<terminated> JamDriver2_2511532002 [Java Application] C:\Users\user\.p2\pool\plugins\org.eclipse.justj.op
=== Program Driver Objek Jam ===

 --- Input Jam 1 ---
Masukan Jam (0-23)   :
18
Masukan Menit (0-59) :
30
Masukan Detik (0-59) :
59

 --- Input Jam 2 ---
Masukan Jam (0-23)   :
22
Masukan Menit (0-59) :
04
Masukan Detik (0-59) :
06

 ---  Hasil Operasi
 Jam 1 (String)        : 18: 30:59
 Jam 2 (String)        : 22: 04:06
 Jam 1 dalam detik     : 66659
 Jam 2 dalam detik     : 79446
Status       : Jam 1 ebih awal (sebelum) Jam 2
 Durasi  (J1 ke J2)    : 12787 detik
Jam 1 Detik Berikutnya : 18: 31:04
Jam 1 Detik Sebelumnya : 18: 30:00
Hasil J1 + J2          : 16: 35:56

 Program Selesai.
```

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep *Abstract Data Type (ADT)* dapat diterapkan dengan baik dalam pemrograman Java melalui pembuatan *ADT* Jam. *ADT* ini mampu merepresentasikan data waktu dalam bentuk jam, menit, dan detik secara terstruktur.

Selain itu, berbagai operasi seperti validasi, konversi ke detik, operasi aritmatika, dan operasi relasional dapat diimplementasikan dengan baik dan berjalan sesuai dengan spesifikasi. Penggunaan driver program juga membantu dalam menguji seluruh fungsi yang telah dibuat, sehingga dapat dipastikan bahwa program bekerja dengan benar.

3.2 Saran

Saran untuk praktikum selanjutnya adalah agar mahasiswa lebih teliti dalam memahami penulisan *syntax* menghindari kesalahan atau eror, terutama dalam *typo* atau salah ketik *syntax.*

DAFTAR PUSTAKA

[1] Inggriani Liem, “Diktat Struktur Data,” Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.